Fuente original: https://algoritmos-rw.github.io/tda_bg/material/guias/greedy/

Greedy

Contenidos

1. Ejercicio resuelto
2. Ejercicios propuestos

Ejercicio resuelto

Enunciado

Implementar un algoritmo greedy que permita obtener el Dominating Set mínimo (es decir, que contenga la menor cantidad de vértices) para el caso de un árbol (en el contexto de teoría de grafos, no un árbol binario). Indicar y justificar la complejidad del algoritmo implementado. Justificar por qué se trata de un algoritmo greedy. Indicar si el algoritmo siempre da solución óptima. Si lo es, explicar detalladamente, sino dar un contraejemplo.

Consideración

Dado que es posible que para cuando estén realizando los ejercicios de Greedy aún no hayan visto sobre el problema de Dominating Set, aquí damos su definición:

Un dominating set de un grafo es un subconjunto de vértices de dicho grafo tal que, para cada vértice del grafo, dicho vértice pertenece al dominating set, o bien alguno de sus adyacentes pertenece al dominating set.

Solución

Una primera idea que puede surgir es justamente pensar en los grados de los vértices. Tal como vimos Además, sabiendo que todo árbol de al menos 2 vértices tiene al menos 2 hojas que (al igual que el resto de vértices en el árbol) deben “ser dominadas”, quedaría una decisión que tomar: o bien incluir la hoja en el dominating set, o a sus respectivos adyacentes. Dado que queremos minimizar la cantidad de vértices en el dominating set, nos conviene poner el vértice que “más pueda dominar”. En particular, es claro que conviene el adyacente a la hoja, que puede cubrir más de uno.

Acá pueden surgir varios errores. El error más común es el siguiente: ordenar de menor a mayor grado, agregando el vértice y “anulando los que siguen”. Esto es un error porque estamos justamente agarrando al vértice contrario al que nos conviene (de óptimo local, nada). Ok, corrijamos eso, podríamos decir que ordenamos de menor a mayor grado, y en ese orden nos quedamos con el adyacente a dicho vértice. El problema ahora es que puede haber vértices ya dominados que generan la falsa ilusión de que un vértice estaría dominando efectivamente la mayor cantidad posible de vértices por ser sus adyacentes. Entonces, por lo mencionado, tenemos que considerar cuáles vértices ya están siendo dominados (es decir, quienes o bien están en el conjunto o tienen un ayacente en el conjunto). Esto empieza a tener algo de sentido. ¿El problema? Esto tiene un error conceptual gravísimo: técnicamente, no es un algoritmo greedy. Si quien lee dice “pero si ordené!”, ordenar no hace que un algoritmo sea greedy. Ordenar es únicamente una optimización que encontramos cuando el valor por el que vamos a ir buscando “el siguiente mínimo/máximo” no se ve alterado. Pero en este caso, eso no es cierto. Acá nos interesa ver por grado, a medida que, básicamente, estamos eliminando vértices del grafo para dejar de considerarlos. Esto altera el grado que debo considerar, por lo que el estado cambió y debo buscar el nuevo mínimo. El tema aquí es que haber ordenado y no considerar las modificaciones quita el elemento “local”. Cuando decimos que buscamos un óptimo local, justamente es local al estado de la iteración como la tenemos, no necesariamente esto está relacionado al estado original. Hay algoritmos donde el estado original va a permanecer para servirnos como óptimos locales (ejemplo, el algoritmo de Kruskal), pero en otros no es el caso y hay que ir considerando las actualizaciones (ejemplos, Dijkstra y Huffman). Por supuesto esto demora más, pero… es óptimo. La otra alternativa no es siquiera óptima, además de no ser enteramente greedy.

Para darles un contraejemplo, supongamos un grafo:

```
A - B - D - F
    |   |
    C   E
```

En este caso, inicialmente cubriremos B (o D), eliminando a A, C y D en el camino, dejándonos con E y F, los cuáles ambos deberían ser incluídos, dejándonos con un Dominating Set de 3 vértices cuando el óptimo es de 2 (B y D). Podríamos seguir agregando condiciones, pero hay que tener cuidado de no decir “bueno, ahora intento considerar una opción y otra”, cuando un algoritmo greedy aplica una simple regla para buscar el óptimo local.

Entonces, volvemos a plantear la solución realmente greedy:

- Buscamos un vértice de grado 1. A su adyacente lo agregamos al dominating set.
- Agregamos al conjunto de vértices dominados al vértice que agregamos al dominating set y a todos sus adyacentes
- Eliminamos del grafo a: el vértice de grado 1, al agregado al dominating set, y también a todos los adyacentes al vértice agregado que sean hojas (es decir, este podría estar vinculado a varias hojas), o que por eliminar a este pasaran a ser hojas (ya dominadas, no traen otro beneficio).
- Volvemos a buscar algún vértice de grado 1 aplicando la misma lógica, hasta que no queden vértices en el grafo, o solo queden vértices de grado 0
- Finalmente, si no quedan vertices de grado 1 pero el grafo aún tiene vertices (necesariamente todos los vértices que quedan tienen grado 0), agregamos al dominating set a los vértices del grafo que aún no están en el conjunto de dominados.

```
def dominating_set_arbol(grafo):
    ds = set()
    dominados = set()
    while len(grafo) > 0:
        v = obtener_vertice_grado_1(grafo)
        if v is None:
            break

        w = grafo.adyacentes(v)[0]
        ds.add(w)
        dominados.add(w)
        for x in grafo.adyacentes(w):
            dominados.add(x)
            if len(grafo.adyacentes(x)) <= 2:
                grafo.borrar_vertice(x)
        grafo.borrar_vertice(w)

    for v in grafo:
        if v not in dominados:
            ds.add(v)

    return ds

def obtener_vertice_grado_1(grafo):
    for v in grafo:
        if len(grafo.adyacentes(v)) == 1:
            return v
```

Hay otras formas de implementar esto (buscar en cada iteración a los vértices de grado 1 y aplicar esta lógica), que en sí sería lo mismo. También se puede obviar “eliminar” los vértices y simplemente ir modificando un diccionario con grados y cosas por el estilo.

La complejidad de este algoritmo es $\mathcal{O}(n^2)$, por la búsqueda de vértices de grado 1 en cada iteración. El algoritmo es greedy ya que iterativamente, a través de una regla sencilla, se busca aquel vértice que me maximiza la cantidad de hojas a dominar, las cuales son fáciles de detectar lo que conviene. Siendo este el óptimo local. Al saber que alguno de estos tiene que ir si o si, y que al eliminar una hoja siempre vamos a seguir teniendo un árbol ($\rightarrow$ va a tener al menos 2 hojas), entonces esta lógica siempre será correcta. Y por esto mismo el algoritmo será óptimo (no pedimos una demostración formal en el parcial, pero la idea viene justamente por ese lado, y usando el absurdo).

Alguien podría pensar que al no borrar todos los vértices adyacentes al que estamos agregando cometemos un error porque no estamos considerando que este ya se encuentra dominado. Esto es un error: estamos justamente considerando que a pesar de estar dominado, esto no hace que no convenga nunca incluirlo (mirar el contraejemplo anterior). Si no se lo elimina es porque tiene otro adyacente que aún falta por dominar.

Ejercicios propuestos

1. (★) Explicar por qué el Algoritmo de Kruskal (para obtener el MST de un grafo no dirigido) es un Algoritmo Greedy.

2. (★) Explicar por qué el Algoritmo de Prim (para obtener el MST de un grafo no dirigido) es un Algoritmo Greedy.

3. (★) Explicar por qué el Algoritmo de Dijkstra (para obtener caminos mínimos desde un vértice, en un grafo con pesos positivos) es un Algoritmo Greedy.

4. (★★) Dada un aula/sala donde se pueden dar charlas. Las charlas tienen horario de inicio y fin. Implementar un algoritmo Greedy que reciba el arreglo de los horarios de las charlas, representando en tuplas los horarios de inicios de las charlas, y sus horarios de fin, e indique cuáles son las charlas a dar para maximizar la cantidad total de charlas. Indicar y justificar la complejidad del algoritmo implementado.

5. (★) Realizar un seguimiento de aplicar el Algoritmo de Huffman al texto `"PRETERINTENCIONALIDAD"`, indicando el binario resultante de comprimirlo. ¿Por qué se trata de un algoritmo Greedy? Justificar

6. (★) Se tiene un sistema monetario (ejemplo, el nuestro). Se quiere dar "cambio" de una determinada cantidad de plata. Implementar un algoritmo Greedy que devuelva el cambio pedido, usando la mínima cantidad de monedas/billetes. El algoritmo recibirá un arreglo de valores del sistema monetario, y la cantidad de cambio objetivo a dar, y debe devolver qué monedas/billetes deben ser utilizados para minimizar la cantidad total utilizda. Indicar y justificar la complejidad del algoritmo implementado. ¿El algoritmo implementado encuentra siempre la solución óptima? Justificar si es óptimo, o dar un contraejemplo. ¿Por qué se trata de un algoritmo Greedy? Justificar

7. (★★) Tenemos unos productos dados por un arreglo R, donde $R[i]$ nos dice el precio del producto. Cada día podemos y debemos comprar uno (y sólo uno) de los productos, pero vivimos en una era de inflación y los precios aumentan todo el tiempo. El precio del producto i el día j es $R[i]^{j+1}$ (j comenzando en 0). Implementar un algoritmo greedy que nos indique el precio mínimo al que podemos comprar todos los productos. Indicar y justificar la complejidad del algoritmo implementado. ¿El algoritmo implementado encuentra siempre la solución óptima? Justificar. ¿Por qué se trata de un algoritmo Greedy? Justificar ¿Qué modificaciones se deben realizar para un estado de deflación, con productos que bajan de precio todo el tiempo?

8. (★★) Tenemos una mochila con una capacidad W. Hay elementos a guardar, cada uno tiene un valor, y un peso que ocupa de la capacidad total. Queremos maximizar el valor de lo que llevamos sin exceder la capacidad. Implementar un algoritmo Greedy que, reciba dos arreglos de valores y pesos de los elementos, y devuelva qué elementos deben ser guardados para maximizar la ganancia total. Indicar y justificar la complejidad del algoritmo implementado. ¿El algoritmo implementado encuentra siempre la solución óptima? Justificar. ¿Por qué se trata de un algoritmo Greedy? Justificar

   ¿Qué diferencias se perciben si en vez de tener que colocar los elementos completos, se pueden fraccionar para nuestra conveniencia?

9. (★★) Tenemos tareas con una duración y un deadline (fecha límite), pero pueden hacerse en cualquier momento, intentando que se hagan antes del deadline. Una tarea puede completarse luego de su deadline, pero ello tendra una penalización de latencia. Para este problema, buscamos minimizar la latencia máxima en el que las tareas se ejecuten. Es decir, dados los arreglos de: T tiempo de duraciones de las tareas y L representando al deadline de cada tarea, si definimos que una tarea i empieza en S_i, entonces termina en $F_i = S_i + T_i$, y su latencia es $L_i = F_i - D_i$ (si $F_i > D_i$, sino 0). Nuestra latencia máxima será aquella i que maximice el valor L_i. Implementar un algoritmo que defina en qué orden deben realizarse las tareas, sabiendo que al terminar una tarea se puede empezar la siguiente. Indicar y justificar la complejidad del algoritmo implementado.

   Devolver un arreglo de tuplas, una tupla por tarea, en el orden en que deben ser realizadas, y que cada tupla indique: (el tiempo T_i de la tarea i, y la latencia resultante L_i de esa tarea).

   ¿El algoritmo implementado encuentra siempre la solución óptima? Justificar. ¿Por qué se trata de un algoritmo Greedy? Justificar

10. (★) Una ruta tiene un conjunto de bifurcaciones para acceder a diferentes pueblos. El listado (ordenado por nombre del pueblo) contiene el número de kilómetro donde está ubicada cada una. Se desea ubicar la menor cantidad de patrullas policiales (en las bifurcaciones) de tal forma que no haya bifurcaciones con vigilancia a más de 50 km. Justificar que la solución es óptima. Indicar y justificar la complejidad del algoritmo implementado. Ejemplo:

Ciudad	Bifurcación
Castelli	185
Gral Guido	242
Lezama	156
Maipú	270
Sevigne	194

    Si pongo un patrullero en la bifurcación de Lezama, cubro Castelli y Sevigne. Pero no Gral Guido y Maipú. Necesitaría en ese caso, poner otro. Agrego otro patrullero en Gral Guido. Con eso tengo 2 móviles policiales en bifurcaciones que cubren todas los accesos a todas las ciudades con distancia menor a 50km.

    En un caso alternativo donde solamente se consideren las bifurcaciones de Castelli, Gral Guido y Sevigne, la única solución óptima sería colocar un móvil policial en Sevigne.

11. (★★) Las bolsas de un supermercado se cobran por separado y soportan hasta un peso máximo P, por encima del cual se rompen. Implementar un algoritmo greedy que, teniendo una lista de pesos de n productos comprados, encuentre la mejor forma de distribuir los productos en la menor cantidad posible de bolsas. Realizar el seguimiento del algoritmo propuesto para bolsas con peso máximo 5 y para una lista con los pesos: `[ 4, 2, 1, 3, 5 ]`. ¿El algoritmo implementado encuentra siempre la solución óptima? Justificar. Indicar y justificar la complejidad del algoritmo implementado.

12. (★★) Trabajamos para el mafioso Arnook, que es quien tiene la máxima influencia y poder en la zona costera de Ciudad República. Allí reina el caos y la delincuencia, a tal punto que quien termina organizando las pequeñas mafias locales no es otro sino Arnook. En particular, nos vamos a centrar en unos pedidos que recibe de parte de dichos grupos por el control de diferentes kilómetros de la ruta costera. Cada pequeña mafia le pide a Arnook control sobre un rango de kilómetros (por ejemplo, la mafia nro 1 le pide del kilómetro 1 al 3.5, la mafia 2 le pide del 3.3333 al 8, etc. . .). Si hay una mafia tomando control de algún determinado kilómetro, no puede haber otra haciendo lo mismo (es decir, no pueden solaparse). Cada mafia pide por *un rango específico*. Arnook no cobra por kilómetraje sino por "otorgar el permiso",
indistintamente de los kilómetros pedidos. Ahora bien, esto es una mafia, no una ONG, y no debe rendir cuentas con nadie, así que lo único que es de interés es maximizar la cantidad de permisos otorgados (asegurándose de no otorgarle algún lugar a dos mafias diferentes). Implementar un algoritmo Greedy que reciba los rangos de kilómetros pedidos por cada mafia, y determine a cuáles se les otorgará control, de forma que no hayan dos mafias ocupando mismo territorio, y a su vez maximizando la cantidad de pedidos otorgados. Indicar y justificar la complejidad del algoritmo implementado. Justificar por qué el algoritmo planteado es Greedy. ¿El algoritmo da la solución óptima siempre?

13. (★★) Tenemos una ruta recta muy larga, de K kilómetros, sobre la cual hay casas dispersas. En dichas casas vive gente que usa mucho sus celulares. El intendente a cargo la ruta debe renovar por completo el sistema de antenas, teniendo que construir sobre la ruta nuevas antenas. Cada antena tiene un rango de cobertura de R kilómetros (valor constante conocido). Implementar un algoritmo Greedy que reciba **las ubicaciones de las casas**, en número de kilómetro sobre esta ruta (números *reales positivos*) **desordenadas**, y devuelva los kilómetros sobre los que debemos construir las antenas para que todas las casas tengan cobertura, y se construya para esto la menor cantidad de antenas posibles. Indicar y justificar la complejidad del algoritmo implementado. Justificar por qué se trata de un algoritmo greedy. ¿El algoritmo da la solución óptima siempre?

14. (★★) Se tiene una matriz donde en cada celda hay submarinos, o no, y se quiere poner faros para iluminarlos a todos. Implementar un algoritmo **Greedy** que dé la cantidad mínima de faros que se necesitan para que todos los submarinos queden iluminados, siendo que cada faro ilumina su celda y además todas las adyacentes (incluyendo las diagonales), y las directamente adyacentes a estas (es decir, un "radio de 2 celdas"). Indicar y justificar la complejidad del algoritmo implementado. ¿El algoritmo implementado da siempre la solución óptima? Justificar

15. (★★) Se tiene una colección de n libros con diferentes espesores, que pueden estar entre 1 y n (valores no necesariamente enteros). Tu objetivo es guardar esos libros en la menor cantidad de cajas. Todas las cajas disponibles son de la misma capacidad L (se asegura que $L \geq n$). Obviamente, no podés partir un libro para que vaya en múltiples cajas, pero sí podés poner múltiples libros en una misma caja, siempre y cuando los espesores no superen esa capacidad L. Implementar un algoritmo Greedy que obtenga las cajas, tal que se minimicen la cantidad de cajas a utilizar. Indicar y justificar la complejidad del algoritmo implementado. Justificar por qué se trata de un algoritmo greedy. ¿El algoritmo propuesto encuentra siempre la solución óptima? Justificar. ¿Qué cambios aplicarías si supieras que los espesores sólo fueran números enteros de un rango acotado? Describir cómo afecta a la complejidad, y a su optimalidad.

16. (★★★) El club de Amigos de Siempre prepara una cena en sus instalaciones en la que desea invitar a la máxima cantidad de sus n socios. Sin embargo por protocolo cada persona invitada debe cumplir un requisito: Sólo puede ser invitada si conoce a al menos otras 4 personas invitadas.

    a. Nos solicitan seleccionar el mayor número posible de invitados. Proponer una estrategia greedy óptima para resolver el problema.

    b. Los organizadores desean que cada invitado pueda conocer nuevas personas. Por lo que nos solicitan que adicionemos una nueva restricción a la invitación: Sólo puede asistir si NO conoce al menos otras 4 personas invitadas. Modifique su propuesta para satisfacer esta nueva solución.

17. (★★) Imaginá que estamos organizando un torneo de guardias en un castillo. El castillo tiene un suelo dividido en una cuadrícula de tamaño `n x m`, y cada celda puede estar ocupada por un guardia o estar vacía. Los guardias tienen la habilidad de vigilar todas las celdas adyacentes a su posición, incluidas las diagonales, es decir, pueden ver las celdas vecinas que están justo al lado, arriba, abajo, a la izquierda, a la derecha o en las esquinas.

    Se nos pide colocar la mayor cantidad posible de guardias en el castillo sin que ninguno pueda vigilar a otro. Esto significa que no podemos colocar dos guardias en celdas adyacentes, ya que estarían vigilándose mutuamente.

    Implementar un algoritmo **greedy** que permita colocar el mayor número posible de guardias en el castillo sin que se vigilen entre sí. Indicar y justificar la complejidad del algoritmo. Indicar por qué se trata, en efecto, de un algoritmo greedy. El algorimto, ¿es óptimo? si lo es, justificar brevemente, sino dar un contraejemplo.